第一章：算法与基本原理

> ***阅读说明**: 本章系统梳理 LLM Post-Training 的完整技术栈。第一节介绍大模型训练三阶段（Pre / Mid / Post-Training）的全景，第二节将 Post-Training 的两大范式——RLHF 与 RLVR——做一站式概览（含 SFT、RM、DPO 等基础知识），后续各节深入剖析 PPO → GRPO → DAPO → VAPO → CISPO → GSPO → SAPO 的算法演进，包含完整公式推导。涉及难以理解的概念时，会用 「📖 **初学者补充**」 模块单独讲解。*

目录

- 一、大模型训练全景：Pre-Training / Mid-Training / Post-Training
- 二、Post-Training 两大范式：RLHF 与 RLVR
- 三、DPO — 离线偏好优化
- 四、PPO — 经典策略优化基础
- 五、GRPO — RLVR 的奠基算法
- 六、DAPO — 工业级 RLVR 工程优化
- 七、VAPO — Value-Based 路线的回归与修正
- 八、CISPO — 裁剪 IS 权重而非目标函数
- 九、GSPO — 序列级策略优化
- 十、SAPO — 连续信任域的软门控
- 十一、全部算法公式速查表
- 十二、演进逻辑总结

一、大模型训练全景：Pre-Training / Mid-Training / Post-Training

近年来，大语言模型（LLM）的训练流程逐渐形成了三阶段共识。理解每个阶段的定位是理解 Post-Training 的前提。

> ***参考文献**:*
>
> - *A Survey of Post-training for Large Language Models (arXiv:2503.06072, 2025) — 目前最全面的 Post-Training 综述*
> - *A Survey on LLM Mid-Training (arXiv:2510.23081, 2025) — Mid-Training 专项综述*
> - *The Llama 3 Herd of Models (arXiv:2407.21783, 2024) — 首个系统展示三阶段完整流程的工业级报告*

1.1 三阶段全景

```
┌──────────────────────────────────────────────────────────────────────────┐
│  阶段 1: Pre-Training（预训练）                                            │
│    目标: 语言建模 — 学习语言的统计规律和世界知识                            │
│    数据: 万亿级 token 的大规模无标注语料                                    │
│    方法: Next-Token Prediction（自回归语言建模）                            │
│    产出: Base Model（基座模型）                                             │
│    算力: 整个训练流程的 ~95%+ 算力消耗在此阶段                              │
├──────────────────────────────────────────────────────────────────────────┤
```

```
│  阶段 2: Mid-Training (中间训练) [可选]                                    │
│    目标: 能力增强 — 在特定维度上补强基座模型                              │
│    数据: 十亿至千亿级 token 的高质量定向语料                              │
│    方法: 继续预训练 (Continued Pre-Training), 仍以 NTP 为主               │
│    产出: Enhanced Base Model                                              │
│    典型任务: 长上下文扩展、领域适配、多语言增强、代码能力补强             │
├──────────────────────────────────────────────────────────────────────────┤
│  阶段 3: Post-Training (后训练)                                           │
│    目标: 对齐与能力释放 — 让模型"有用、安全、准确"地服务用户              │
│    数据: 万至百万级标注数据 + 在线 RL 采样                                │
│    方法: SFT, RLHF, DPO, RLVR 等                                          │
│    产出: Chat / Instruct Model (最终部署模型)                             │
└──────────────────────────────────────────────────────────────────────────┘
```

1.2 Pre-Training：从数据中学习世界知识

预训练是整个流程的基石。通过在万亿级 token 上做 Next-Token Prediction，模型学习到：

- **语言的统计规律**: 语法、搭配、篇章结构
- **世界知识**: 事实、常识、因果关系
- **推理的"种子"**: 模式匹配、简单逻辑链（后续 RL 将这些种子"释放"为成熟的推理能力）

预训练的核心损失函数：

$$ \mathcal{L}_{\text{PT}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{t=1}^{|x|} \log p_\theta(x_t \mid x_{<t}) \right] $$

预训练消耗了整个训练流程 **95% 以上的算力**，是最昂贵的阶段。

1.3 Mid-Training：定向能力增强

Mid-Training（中间训练）是近年逐渐被明确定义的阶段，其目标是**在不破坏预训练知识的前提下，定向增强基座模型的特定能力**。

> 📖 ***概念澄清***: *Mid-Training 有时也被称为 Continued Pre-Training (CPT)、Post Pre-Training 或 Domain Adaptation。arXiv:2510.23081 的综述将其统一定义为"在预训练之后、Post-Training 之前，使用中等规模高质量数据的继续训练阶段"。*

Mid-Training 的典型任务：

任务	描述	代表模型
长上下文扩展	将上下文窗口从 8K/32K 扩展到 128K/1M+	LLaMA 3.1 (128K), Qwen2.5-1M (1M)
领域适配	在医学/法律/金融等垂直语料上继续训练	各类行业大模型
多语言增强	补充特定语言的高质量语料	Qwen 系列的多语言增强
代码能力补强	在高质量代码语料上继续训练	DeepSeek-Coder

多模态融合	引入图像/视频/音频理解能力	LLaMA 3.2, Gemma 3

Mid-Training 的核心特征：

1. **仍以语言建模为主要目标**——与 Pre-Training 相同的 NTP 损失
2. **数据规模居中**——比预训练少 1-2 个数量级（十亿～千亿 token），但远多于 Post-Training
3. **不涉及对齐**——不关心"有用、安全、准确"，只关心能力本身

1.4 Post-Training：对齐与能力释放

Post-Training 是将"会说话的机器"变为"有用的助手"的关键步骤。其核心目标是**对齐（Alignment）**——让模型的行为与人类的意图和价值观对齐。

为什么 Post-Training 如此重要?

基座模型（即使经过 Mid-Training）虽然拥有强大的语言能力和知识储备，但存在严重问题：

问题	表现	原因
不遵循指令	给它一个问题，它可能续写更多问题而非回答	预训练目标是"预测下一个 token"，不是"回答问题"
生成有害内容	可能输出歧视性、暴力、虚假信息	训练数据中包含此类内容
推理能力未释放	蕴含推理"种子"但无法系统运用	预训练以记忆为主，未专门优化推理

Post-Training 的里程碑式发现（InstructGPT, arXiv:2203.02155）：**经过 RLHF 后训练的 1.3B 小模型，在人类评估中胜过 175B 的 GPT-3 基座模型**。这证明了后训练的杠杆效应远超参数量扩展。

1.5 三阶段关系与 2025 年趋势

三个阶段并非孤立的，它们之间存在紧密的相互影响：

```
Pre-Training 决定能力"上限"
    |
    |  Mid-Training 在特定维度上"拉高上限"
    |      |
    |      |  Post-Training "释放"已有能力并"对齐"价值观
    |      |      |
    ▼      ▼      ▼
  知识储备 → 能力增强 → 对齐与部署
```

2025 年的关键趋势：

1. **Mid-Training 和 Post-Training 的边界逐渐模糊**: 例如 DeepSeek-R1 的 Cold Start SFT 既有 SFT 的形式，也有类似 Mid-Training 的功能
2. **Post-Training 的算力占比上升**: 随着 RLVR 范式的成功（DeepSeek-R1, Qwen3），后训练阶段的 RL 训练算力投入显著增加
3. **"轻量 SFT + 重 RL"** 正在成为主流范式（Meta LLaMA 4, Google Gemini 2.5 均确认了此趋势）

二、Post-Training 两大范式：RLHF 与 RLVR

Post-Training 的核心问题是：**如何告诉模型"什么是好的回答"？** 对这个问题的不同回答，形成了两大范式。

2.1 RLHF 范式：从人类反馈中学习

RLHF (Reinforcement Learning from Human Feedback) 的核心洞察是：**人类虽然很难写出完美的回答，但很擅长比较两个回答的好坏。**

经典三阶段流程

```
阶段 1: SFT (Supervised Fine-Tuning)
   收集 (prompt, demonstration) 数据 → 微调基座模型
   损失: L_SFT = -E[Σ log π(y_t | x, y_{<t})]  (只对回答计算损失)
   作用: 学会基本的指令遵循、对话格式

阶段 2: 奖励模型训练 (Reward Model)
   收集 (prompt, response_A, response_B, preference) 数据
   → 用 Bradley-Terry 模型训练 RM 学习人类偏好
   损失: L_RM = -E[log σ(r(x,y_w) - r(x,y_l))]

阶段 3: RL 优化
   用 PPO 算法，以 RM 分数为奖励，优化策略模型
   目标: max E[r(x,y)] - β·KL[π_θ || π_ref]
```

RL 优化目标

$$ \max_{\pi_\theta} ; \mathbb{E}{x |sim |mathcal{D},, y |sim |pi_|theta(|cdot|x)} |left[r_|varphi(x, y) |right] - |beta |cdot |text{KL}|left[|pi_|theta(|cdot|x) ,|, |pi{\text{ref}}(\cdot|x)\right] $$

其中 KL 惩罚防止模型为了追求高奖励而偏离正常语言分布太远（reward hacking）。

奖励模型的已知问题

问题	表现	应对策略
Reward Hacking	模型学会"讨好" RM 而非真正提升质量	KL 惩罚、多 RM 集成 (WARM)
长度偏差	RM 倾向于给更长回答更高分	长度归一化、显式长度惩罚
泛化不足	RM 在训练域外不可靠	迭代式 RM 更新 (LLaMA 3.1 的 6 轮迭代)
分布偏移	策略更新后 RM 面对的输入分布变化	在线 RM 更新、RLVR 绕过 RM

> 📖 ***初学者补充：SFT 为什么是必要的第一步？***

> *SFT 教会模型两件事：(1) 输出格式——学会"问题-回答"的对话结构而非无限续写；(2) 基本能力——遵循指令、使用工具、控制输出风格。没有 SFT，直接做 RL 会遇到严重的格式混乱问题（R1-Zero 的教训：语言混杂、输出不可读）。但最新趋势表明 SFT 应该"轻量化"——过重的 SFT 反而会限制 RL 的探索空间。*

2.2 RLVR 范式：基于可验证奖励的强化学习

RLVR (RL with Verifiable Rewards) 由 DeepSeek-R1 (arXiv:2501.12948) 开创，其核心思想极其简单：**对于有确定答案的任务，直接用答案正确性作为奖励，完全绕过 RM。**

$$ R(\hat{y}, y) = \begin{cases} +1, & \text{is_equivalent}(\hat{y}, y) \ -1, & \text{otherwise} \end{cases} $$

RLHF vs RLVR 对比

维度	RLHF	RLVR
奖励来源	训练的 RM（近似人类偏好）	规则判定（答案匹配、代码测试通过）
Reward Hacking 风险	高（RM 有漏洞）	**极低**（答案对就是对）
适用任务	通用（对话、创作、对齐）	**仅限有确定答案的任务**（数学、代码、逻辑）
数据需求	需要人类偏好标注	只需 (题目, 标准答案) 对
核心算法	PPO + RM	GRPO, DAPO, VAPO 等
代表模型	InstructGPT, LLaMA 2, Claude	DeepSeek-R1, Qwen3, Seed1.5-Thinking

为什么 RLVR 在推理任务上取得了突破?

1. **零 Reward Hacking**: 数学题答案不存在"讨好"的空间，1+1=2 就是 2
2. **信号清晰**: 对/错的二元反馈虽然稀疏，但完全没有噪声
3. **可大规模扩展**: 数学题可以无限生成，不需要人类标注
4. **涌现能力**: DeepSeek R1-Zero 证明了纯 RLVR 就能让模型涌现出自我反思、纠错等高级推理行为

2025 年的收敛范式

尽管各家的具体实现不同，Post-Training 的 Pipeline 在 2025-2026 年逐渐趋同：

```
通用范式（2025-2026）:
  1. Lightweight SFT — 少量高质量数据，学会格式和基本能力
  2. Reasoning RL（RLVR）— 数学/代码/逻辑，基于规则奖励
  3. [可选] Agentic RL — 工具使用/多轮交互
  4. General RL（RLHF）— 全场景对齐（有用性 + 安全性）
```

三、DPO — 离线偏好优化

> ***论文**: Direct Preference Optimization: Your Language Model is Secretly a Reward Model (Rafailov et al., 2023) **arXiv**: 2305.18290*

DPO 是 RLHF 的一个重要简化变体：**绕过 RM，直接从偏好数据优化策略模型**。

3.1 损失函数

$$ \mathcal{L}{|text{DPO}}(|theta) = -|mathbb{E}{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma!\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] $$

DPO 要求模型让好回答 y_w 与坏回答 y_l 之间的对数概率比差距足够大。优点是不需要 RM、训练稳定、易于实现；缺点是依赖离线数据，存在分布偏移问题。

3.2 DPO 的实际定位

DPO 在实际工业应用中的定位越来越清晰：

- **通用对齐**: LLaMA 3.1 大规模对比后发现 DPO 在一般对齐任务上可优于 PPO（但这是有争议的，DeepSeek/ByteDance 经验相反）
- **推理任务**: 在复杂数学/代码推理上，在线 RL（GRPO/DAPO/VAPO）**明显优于** DPO
- **最新趋势**: LLaMA 4 将 DPO 从主力降级为"轻量后处理"步骤，核心推理提升交由 Online RL

> *DPO 更多的技术细节和变体（KTO, ORPO, IPO 等）可参考综述 arXiv:2503.06072。*

四、PPO — 经典策略优化基础

> ***论文**: Proximal Policy Optimization Algorithms (Schulman et al., 2017) **地位**: 整条演进路线的起点，ChatGPT/InstructGPT 的核心算法*

4.1 核心思想

PPO 的目标是：在更新策略时，不要走得太远——通过一个**裁剪的替代目标函数 (clipped surrogate objective)** 平衡探索与稳定。

4.2 核心公式

$$ \mathcal{J}{|text{PPO}}(|theta) = |mathbb{E}{q \sim P(Q),, o \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \min \left(r_t(\theta) \hat{A}_t, ; \text{clip}\big(r_t(\theta),, 1-\varepsilon,, 1+\varepsilon\big) \hat{A}_t \right) \right] $$

其中：

- **重要性采样比率**: $r_t(\theta) = \frac{\pi_\theta(o_t \mid q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t \mid q, o_{<t})}$
- **优势函数** $\hat{A}_t$ 由 **GAE (Generalized Advantage Estimation)** 计算：

$$ \hat{A}t^{|text{GAE}(|gamma, |lambda)} = |sum{l=0}^{T-t-1} (\gamma \lambda)^l , \delta_{t+l}, \quad \delta_l = R_l + \gamma , V_\psi(s_{l+1}) - V_\psi(s_l) $$

- V_ψ 是**价值网络 (Critic)**，一个与策略模型同等规模的独立网络
- ε 通常取 0.2

> 📖 ***初学者补充：PPO 的三个关键概念***
>
> 1. ***策略 (Policy)**: 在 LLM 中，策略 π_θ 就是模型本身——给定问题 q 和已生成的 token $o_{<t}$，输出下一个 token o_t 的概率分布*
> 2. ***重要性采样比率 $r_t(\theta)$**: 用旧策略采样的数据更新新策略时的偏差矫正系数。$r_t = 1$ 表示新旧一致，$r_t > 1$ 表示新策略更倾向于生成这个 token*
> 3. ***优势函数 $\hat{A}_t$**: "这个动作比平均水平好多少"。$\hat{A}_t > 0$ 表示该动作超出预期（鼓励），< 0 表示低于预期（抑制）*

4.3 裁剪机制

`min(r·A, clip(r)·A)` 的效果是**双向限制**：

情况	$\hat{A}_t > 0$（好动作）	$\hat{A}_t < 0$（坏动作）
r_t 太大	裁剪到 $1+\varepsilon$，防止过度增加	不裁剪，允许自由减少
r_t 太小	不裁剪，允许自由增加	裁剪到 $1-\varepsilon$，防止过度减少

4.4 PPO 在 LLM 场景的问题

PPO 最初为游戏/机器人设计，在 LLM 场景中暴露了核心问题：

1. **需要 4 个模型同时在 GPU 上**: 策略模型 + 参考模型 + 奖励模型 + 价值网络（Critic），显存开销巨大
2. **Critic 训练困难**: LLM 的奖励通常只在最后一个 token 给出（稀疏奖励），但 Critic 需要在每个 token 位置给出准确的价值估计
3. **长序列中 Critic 不准**: 对于长 Chain-of-Thought (CoT) 推理，Critic 估计的误差会累积

这些问题直接催生了 GRPO 的设计。

五、GRPO — RLVR 的奠基算法

> ***论文**: DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (Shao et al., 2024) **arXiv**: 2402.03300 **地位**: Value-Free RL 路线的开创者，**RLVR 范式的确立**，DeepSeek-R1 的核心算法*

5.1 核心思想

GRPO 的核心洞察：**不需要价值网络（Critic），也不需要训练奖励模型**。对同一个问题采样一组（Group）回答，用组内的相对奖励来估计优势，用规则判定（答案是否正确）来提供奖励。

GRPO = Value-Free + RM-Free，这正是 RLVR 范式得以成立的技术基础。

5.2 核心公式

GRPO 目标函数:

$$ \mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim P(Q),, {o_i}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min!\left(r_{i,t}(\theta), \hat{A}_{i,t},, \text{clip}!\left(r_{i,t}(\theta),, 1!-!\varepsilon,,

1!+!\varepsilon\right) \hat{A}*{i,t} |right) - |beta, |mathbb{D}*{\text{KL}}[\pi_\theta | \pi_{\text{ref}}] \right) \right] $$

与 PPO 的关键区别——优势函数的计算方式:

$$ \hat{A}_{i,t} = \tilde{r}_i = \frac{R_i - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})} $$

其中 $\mathbf{R} = {R_1, R_2, \ldots, R_G}$ 是同一问题下 G 个采样回答的奖励集合。在 RLVR 范式下，$R_i \in {+1, -1}$（答案对/错）。

> 📖 ***初学者补充：Group Relative 是什么意思？***
>
> *举个例子：对问题 "求 $|sqrt{144}$"，模型生成 8 个回答（Group Size $G=8$）：*
>
> | *回答* | *正确?* | *奖励 R_i* |
> |---|---|---|
> | *o_1: "12"* | ✅ | *+1* |
> | *o_2: "14"* | ❌ | *-1* |
> | *o_3: "12"* | ✅ | *+1* |
> | *o_4: "11"* | ❌ | *-1* |
> | ... | ... | ... |
>
> *计算：$|text{mean}(|mathbf{R}) = 0$，$|text{std}(|mathbf{R}) = 1$*
>
> - *正确回答：$|hat{A} = (1 - 0)/1 = +1$（鼓励）*
> - *错误回答：$|hat{A} = (-1 - 0)/1 = -1$（抑制）*
>
> *这就是 "Group Relative"：优势是**相对于同组其他回答**的，不需要 Critic 网络来估计。*

GRPO 的 KL 散度估计器（无偏估计）:

$$ \mathbb{D}*{|text{KL}}[|pi_|theta | |pi*{\text{ref}}] = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - 1 $$

5.3 为什么这么设计

设计决策	动机	解决的问题
消除 Critic 网络	Critic 在 LLM 稀疏奖励场景下训练困难	4 模型 → 3 模型，节省 ~25% 显存
消除 RM	数学/代码任务有确定答案，不需要学习的奖励模型	杜绝 Reward Hacking
组内相对奖励	对/错的二元奖励天然适合相对比较	无需绝对准确的价值估计
保留 PPO 裁剪	裁剪仍然是稳定训练的有效手段	防止策略突变

5.4 GRPO 的局限

1. **Hard Clipping 丢失梯度**: 被裁剪的 token 梯度变为 0，某些稀有但重要的推理词（如 "however", "wait"）可能永远学不到
2. **Token-Level IS 高方差**: 逐 token 的重要性采样权重在长序列中波动剧烈
3. **长序列中 Group Baseline 精度不足**: 同一组所有 token 共享同一个优势值，无法区分序列内部不同位置的贡献

> 📖 ***深入理解：为什么 Hard Clipping 会让被裁剪 token 的梯度变为 0？***
>
> *PPO/GRPO 的目标函数对单个 token 的贡献：*
>
> *$$L_t = |min!|big(r_t |hat{A}_t,; |text{clip}(r_t, 1!-!|varepsilon, 1!+!|varepsilon) |cdot |hat{A}_t|big)$$*
>
> *当 $|hat{A}_t > 0$（好 token）且 $r_t > 1 + |varepsilon$ 时（新策略已经大幅增加了这个 token 的概率），$|min$ 取常数项 $(1+|varepsilon) |cdot |hat{A}_t$，这是**关于 $|theta$ 的常数**，所以 $|nabla_|theta L_t = 0$。*
>
> ***PPO 的裁剪等价于一个二值掩码：***
>
> *$$|nabla_|theta L_t = |underbrace{M_t}_{|in {0,1}} |cdot |hat{A}_t |cdot |nabla_|theta |log |pi_|theta(o_t |mid |cdots)$$*
>
> *$$M_t = |begin{cases} 0, & |hat{A}_t > 0 |text{ 且 } r_t > 1+|varepsilon |quad |text{(已经够鼓励了，停止)} | 0, & |hat{A}_t < 0 |text{ 且 } r_t < 1-|varepsilon |quad |text{(已经够抑制了，停止)} | 1, & |text{otherwise} |quad |text{(继续更新)} |end{cases}$$*
>
> ***对稀有推理词的影响**：像 "However" 这样的词在基座模型中概率可能只有 $|pi_{|text{old}} = 0.005$。模型概率增长到 0.01 时，$r_t = 2.0$，远超 1.2，梯度归零——尽管绝对概率只有 1%，离"学会"还差很远。*

六、DAPO — 工业级 RLVR 工程优化

> ***论文**: DAPO: An Open-Source LLM Reinforcement Learning System at Scale (ByteDance Seed & Tsinghua AIR, 2025) **arXiv**: 2503.14476 **地位**: 首个完全开源的大规模 RLVR 系统，AIME 2024 达到 50%*

6.1 核心公式

$$ \mathcal{J}{|text{DAPO}}(|theta) = |mathbb{E}{(q,a)\sim\mathcal{D},, {o_i}{i=1}^G |sim |pi{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \min!\left(r_{i,t}(\theta), \hat{A}{i,t},; |text{clip}!|left(r{i,t}(\theta),, 1!-!\varepsilon_{\text{low}},, 1!+!\varepsilon_{\text{high}}\right) \hat{A}_{i,t} \right) \right] $$

$$ \text{s.t.} \quad 0 < \big|{o_i \mid \text{is_equivalent}(a, o_i)}\big| < G $$

6.2 四大核心改进

改进 ①：Clip-Higher（非对称裁剪）

$$ \text{clip}(r_{i,t}(\theta),; 1 - \varepsilon_{\text{low}},; 1 + \varepsilon_{\text{high}}) $$

实验设定：$\varepsilon_{\text{low}} = 0.2$，$\varepsilon_{\text{high}} = 0.28$。

放宽上界给低概率的"探索性" token 更多增长空间，同时保持下界防止概率坍塌。

改进 ②：Dynamic Sampling（动态采样）

约束条件 $0 < |{o_i \mid \text{is_equivalent}(a, o_i)}| < G$ 过滤掉全对/全错的 prompt（它们的优势全为 0，梯度为 0），确保每个 batch 都有有效梯度。

改进 ③：Token-Level Policy Gradient Loss（全局 Token 归一化）

GRPO 的 sample-level 归一化：

$$ \mathcal{L}_{\text{GRPO}} = \frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} L_{i,t} $$

DAPO 的 token-level 归一化：

$$ \mathcal{L}_{\text{DAPO}} = \frac{1}{\sum_{i=1}^{G} |o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} L_{i,t} $$

Token-level 让每个 token 权重一致，避免长回答中 token 贡献被稀释。消融实验：Token-Level 50.0 vs Sample-Level 43.0（AIME 2024）。

改进 ④：Overlong Reward Shaping（超长输出奖励塑形）

$$ R_{\text{length}}(y) = \begin{cases} 0, & |y| \leq L_{\max} - L_{\text{cache}} \ \frac{(L_{\max} - L_{\text{cache}}) - |y|}{L_{\text{cache}}}, & L_{\max} - L_{\text{cache}} < |y| \leq L_{\max} \ -1, & |y| > L_{\max} \end{cases} $$

软惩罚让模型学到"尽量在限制内完成"，而非"长就是错"。

6.3 移除 KL 散度

DAPO 完全移除了 KL 惩罚项。理由：在长 CoT 推理场景中，模型分布需要大幅偏离初始模型才能涌现复杂推理行为，KL 约束反而是障碍。

七、VAPO — Value-Based 路线的回归与修正

> ***论文**: VAPO: Efficient and Reliable Reinforcement Learning for LLM Reasoning with Verifiable Rewards (ByteDance Seed, 2025) **arXiv**: 2504.05118 **地位**: 跨分支融合算法（PPO + GRPO + DAPO），AIME 2024 达到 60.4%*

7.1 核心论点

"Value-Free（如 GRPO）在短任务上够用，但在**长 CoT 推理**中，Value-Based 有不可替代的优势——只是需要修好 Critic。"

7.2 核心公式

VAPO 总损失函数:

$$ \mathcal{L}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) + \mu \cdot \mathcal{L}_{\text{NLL}}(\theta) \quad (\mu = 0.1) $$

PPO 部分采用 Clip-Higher + Token-Level Loss + GAE（非 Group Relative）。

正样本模仿学习损失 (Positive-Example LM Loss):

$$ \mathcal{L}_{\text{NLL}}(\theta) = -\frac{1}{\sum_{o_i \in \mathcal{T}} |o_i|} \sum_{o_i \in \mathcal{T}} \sum_{t=1}^{|o_i|} \log \pi_\theta(a_t \mid s_t) $$

其中 $\mathcal{T}$ 是当前 batch 中**回答正确的样本集合**。这是一个"自我模仿学习"——对正确回答直接最大化对数似然。去掉此损失，AIME 从 60 降到 54（−6）。

7.3 三大 Critic 修复技术

技术	做法	移除后 AIME 得分
Value Pretraining	RL 开始前用 MC 回报预训练 Critic ~50 步	11（−49，几乎崩溃）
Decoupled GAE	Critic 用 $\lambda=1.0$（无偏），Policy 用 $\lambda=0.95$（低方差）	33（−27）
Length-Adaptive GAE	$\lambda_{\text{policy}} = 1 - \frac{1}{\alpha \cdot l}$，让 TD 误差贡献与序列长度成正比	45（−15）

> 📖 ***初学者补充：为什么 Value Pretraining 如此关键？***
>
> *从奖励模型初始化 Critic 会引入**正向偏置**——RM 被训练在 `<EOS>` 位置打分，对早期 token 给出较低分数。但 Critic 需要在每个 token 给出准确的期望累积回报。没有预训练，Critic 的初始估计完全不靠谱，导致 GAE 计算出的优势全是噪声，RL 从一开始就走向错误方向。*

7.4 技术来源总结

技术	来源	创新/继承
Value Model + GAE	PPO	继承
Group Sampling	GRPO	继承
Clip-Higher, Token-Level Loss, Dynamic Sampling	DAPO	继承
Value Pretraining, Decoupled GAE, Length-Adaptive GAE, Positive-Example LM Loss	**VAPO 原创**	创新

7.5 消融实验汇总

移除的组件	AIME 2024	下降
完整 VAPO	**60.4**	—
移除 Value Pretraining	11	−49

移除 Decoupled GAE	33	−27
移除 Length-Adaptive GAE	45	−15
移除 Clip-Higher	46	−14
移除 Token-Level Loss	53	−7
移除 Positive-Example LM Loss	54	−6
原始 PPO（无任何改进）	5	−55

八、CISPO — 裁剪 IS 权重而非目标函数

> ***论文**: MiniMax-M1: Scaling Test-Time Compute Efficiently with Thinking Models (MiniMax, 2025)* ***arXiv**: 2506.13585* ***地位**: 首次提出"裁剪 IS 权重而非裁剪目标"的范式*

8.1 核心问题

PPO/GRPO 的 Hard Clipping 对推理关键的"分岔词"（如 "However"、"Wait"）特别不友好。这些词初始概率低，$r_{i,t}$ 很快超过裁剪阈值，梯度归零。在 MiniMax 的设置中（每 batch 16 轮 off-policy 更新），这些 token 在**第 1 轮就被裁剪出局**，剩下 15 轮完全没有梯度。

8.2 核心公式

$$ \mathcal{J}*{|text{CISPO}}(|theta) = |mathbb{E} |left[|frac{1}{|sum*{i=1}^{G}|o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \text{sg}!\left(\hat{r}*{i,t}(|theta)|right) |cdot |hat{A}*{i,t} \cdot \log \pi_\theta(o_{i,t} \mid q, o_{i,<t}) \right] $$

其中 $\hat{r}*{i,t}(|theta) = |text{clip}!|left(r*{i,t}(\theta),; 1 - \varepsilon_{\text{low}}^{\text{IS}},; 1 + \varepsilon_{\text{high}}^{\text{IS}}\right)$，$\text{sg}(\cdot)$ 是 stop-gradient。

8.3 PPO vs CISPO 梯度行为

算法	裁剪对象	被裁剪 token 的梯度
PPO/GRPO	目标函数 $r \cdot A$	**= 0**（完全丢失）
CISPO	IS 权重 r（在 sg 内）	**≠ 0**（权重被限制但梯度保留）

CISPO 永远不会将任何 token 的梯度置零。IS 权重被裁剪后通过 $\text{sg}()$ 作为常数标量，但 $\log\pi_\theta(\cdot)$ 始终可微：

$$ \nabla_\theta \mathcal{J}*{|text{CISPO}} |propto |sum*{i,t} \underbrace{\text{sg}(\hat{r}*{i,t})}*{\text{裁剪后的标量权重}} \cdot \hat{A}*{i,t} |cdot |underbrace{|nabla_|theta |log |pi_|theta(o*{i,t} \mid \cdots)}_{\text{始终有梯度}} $$

九、GSPO — 序列级策略优化

> ***论文**: Group Sequence Policy Optimization (Alibaba / Qwen Team, 2025)* ***arXiv**: 2507.18071* ***地位**: 将优化粒度从 Token-Level 提升到 Sequence-Level，应用于 Qwen3*

9.1 核心问题

GRPO 的 Token-Level IS 存在**理论缺陷**：奖励是 sequence-level 的，但 IS 矫正是 token-level 的——优化单元与奖励单元不匹配。更关键的是，token-level IS 权重对"分岔词"造成剧烈波动，在 MoE 模型中尤其不稳定（专家路由切换导致个别 token 的 $r_{i,t}$ 突变）。

9.2 核心公式

$$ \mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^{G} \min!\left(s_i(\theta), \hat{A}_i,; \text{clip}!\left(s_i(\theta),, 1!-!\varepsilon,, 1!+!\varepsilon\right) \hat{A}_i \right) \right] $$

序列级 IS 比率（逐 token 比率的几何平均）：

$$ s_i(\theta) = \left(\frac{\pi_\theta(y_i \mid q)}{\pi_{\theta_{\text{old}}}(y_i \mid q)}\right)^{1/|y_i|} = \exp!\left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid q, y_{i,<t})}\right) $$

9.3 GRPO vs GSPO 梯度对比

- **GRPO**: 同一序列内的不同 token 有**不同权重** $w_{i,t}$
- **GSPO**: 同一序列内的所有 token 有**相同权重** s_i

这消除了 token 间的不一致性，也使得 MoE 模型不受个别 token 专家切换的影响（GRPO 需要 Routing Replay 来解决此问题，GSPO 不需要）。

9.4 极小裁剪范围

由于 $s_i(\theta)$ 经过了 $1/|y_i|$ 归一化，数值范围远比 token-level 紧凑：

$$ \varepsilon_{\text{left}} = 3 \times 10^{-4}, \quad \varepsilon_{\text{right}} = 4 \times 10^{-4} $$

（对比 GRPO 的 $\varepsilon = 0.2 \sim 0.28$，差了三个数量级）

十、SAPO — 连续信任域的软门控

> ***论文**: Soft Adaptive Policy Optimization (Alibaba / Qwen Team, 2025) **arXiv**: 2511.20347 **地位**: 用连续的 Soft Gating 替代所有先前算法的 Hard Clipping，应用于 Qwen3-VL*

10.1 核心问题

所有先前算法都使用某种形式的 Hard Clipping。Hard Clipping 在裁剪边界处存在**不连续的梯度跳变**，难以平衡稳定性与有效学习。

10.2 核心公式

Soft Gating 函数:

$$ f_{i,t}(x) = \sigma!\left(\tau_{i,t} \cdot (x - 1)\right) \cdot \frac{4}{\tau_{i,t}} $$

温度参数根据优势符号自适应：$\tau_{\text{pos}} = 1.0$，$\tau_{\text{neg}} = 1.05$（非对称温度）。

梯度权重: $w_{i,t}(\theta) = 4 \cdot p \cdot (1 - p)$，其中 $p = \sigma(\tau(r-1))$。

- 在 $r = 1$（on-policy）处取最大值 $w = 1$
- 随 r 偏离 1 **平滑对称衰减**（sech^2 形状）
- 永远不为零

10.3 统一视角：三种 Gating 函数

算法	$f(r)$ 形式	边界行为
GRPO	$\min(r, 1+\varepsilon)$	超过阈值后**水平截断**，梯度 = 0
GSPO	$\min(s_i, 1+\varepsilon)$（序列级）	超过阈值后**水平截断**
SAPO	$\frac{4}{\tau}\sigma(\tau(r-1))$	**平滑 S 形饱和**，从不完全截断

10.4 与 GSPO 的联系

论文证明在温和条件下，SAPO 的 token-level soft gate 会**自动近似坍缩为 sequence-level gate**，不需要显式选择 token-level 还是 sequence-level——它在 token-level 定义，但自然地表现出 sequence-level 的行为。

10.5 为什么要非对称温度

- **正优势** ($\hat{A} > 0$)：更新是**聚焦的**——增加采样 token 的 logit，降低其他 token 的 logit，方差小
- **负优势** ($\hat{A} < 0$)：更新是**发散的**——降低采样 token 的 logit，但提升**所有其他 token**（词表大小 ~100K+）的 logit，梯度被分散到大量无关 token，方差大

$\tau_{\text{neg}} > \tau_{\text{pos}}$ 使得负样本的梯度衰减更快，抑制高方差效应。

十一、全部算法公式速查表

算法	目标函数核心	优势函数	IS 粒度	裁剪/门控	范式
PPO	$\min(r_t A_t, \text{clip}(r_t) A_t)$	GAE + Critic	Token	对称硬裁剪	RLHF
GRPO	$\min(r_{i,t} \hat{A}*i, |text{clip}(r*{i,t}) \hat{A}_i)$	Group Relative	Token	对称硬裁剪	**RLVR**
DAPO	$\min(r_{i,t} \hat{A}*i, |text{clip}*{\text{asym}}(r_{i,t}) \hat{A}_i)$	Group Relative	Token	**非对称**硬裁剪	**RLVR**
VAPO	同 DAPO + $\mu \cdot \mathcal{L}_{\text{NLL}}$	GAE + **Adaptive** Critic	Token	非对称硬裁剪	**RLVR**
CISPO	$\text{sg}(\text{clip}(r_{i,t})) \cdot \hat{A}_i \cdot \log\pi$	Group Relative	Token	裁剪**IS 权重**	**RLVR**

GSPO	$\min(s_i \hat{A}_i, \text{clip}(s_i) \hat{A}_i)$	Group Relative	**Sequence**	硬裁剪（极小 ε）	**RLVR**
SAPO	$\frac{4}{\tau}\sigma(\tau(r-1)) \cdot \hat{A}_i$	Group Relative	Token → Seq	**Soft Gating**	**RLVR**

十二、演进逻辑总结：每一步解决了什么问题

```
PPO (RLHF) ──────────── 稳定的策略优化框架
  │ 问题: Critic训练困难、显存占用过高(4模型)、RM有Reward Hacking风险
  │
  ▼
GRPO (RLVR) ─────────── 消除Critic+RM, 用Group Relative + 规则奖励
  │ 问题: Hard Clip丢失梯度 + 熵崩塌 + Token-Level IS高方差
  │
  ├──► DAPO ──────────── 非对称裁剪 + 动态采样 + Token归一化
  │      │                  (工程化解决GRPO实际训练中的一系列问题)
  │      │ 问题: 裁剪本身的梯度丢失问题仍未根治
  │      │
  │      ├──► CISPO ─────── 裁剪IS权重而非目标函数 → 所有token保留梯度
  │      │                  (从根本上解决梯度丢失)
  │      │
  │      └──► GSPO ──────── Token-Level → Sequence-Level
  │             │            (从根本上解决IS高方差 + MoE不稳定)
  │             │ 问题: 硬裁剪在序列级仍有突变
  │             │
  │             └──► SAPO ── Soft Gating替代Hard Clip
  │                            (连续信任域, 非对称温度, 自然坍缩到序列级)
  │
  └──► VAPO ──────────── 回归Value-Based, 修好Critic
                           (Value预训练 + 解耦GAE + 长度自适应GAE)
                           (证明: 修好后的Value-Based在长CoT上更强)
```

其他值得关注的算法：

算法	arXiv	核心贡献
Dr. GRPO	2503.20783	发现 GRPO 存在使错误回答长度增加的优化偏差，提出无偏优化
REINFORCE++	2501.03262	全局优势归一化（跨全局 batch 而非仅组内），指出 GRPO 的局部归一化是有偏估计器
PRIME	2502.01456	通过隐式过程奖励实现在线 PRM 更新，推理 benchmark 平均提升 15.1%

下一章: 第二章：技术报告解读与训练经验